BUILDING ADVANCED AUTONOMOUS AI WITH SAPPHIRE

This guide explains how to build autonomous AI agents in Sapphire -- covering the full pipeline from perception, reasoning, and memory to frontier-scale LLM self-training using ml.distributed.

1. The Autonomous AI Pipeline

Sapphire autonomy pipeline:

Perception --> Memory and DAG Planning --> Autonomous Training (ml.distributed) --> Tools and Execution

An agent can perceive (os.system_info), plan multi-step tasks (agent.memory, DAG), self-train at frontier scale (ml.distributed 5D solver), and act (ai.prompt, fs, http).

2. Agent Memory and Recall

```
agent.memory.remember("user_goal", "Train a 70B LLM");
let goal = agent.memory.recall("user_goal");
print("Goal: {goal}");
```

3. ReAct Autonomous Loop

```
let result = agent.autonomy.run_loop(
    "Analyze system health and fix critical issues",
    max_steps=10
);
print("Loop outcome: {result}");
```

4. Autonomous AI Bot (5-line example)

```
fn main() {
    let stats = os.system_info();
    let opinion = ai.prompt("RAM at {stats.ram_percent}%. Status?");
    print("AI: {opinion}");
    os.notify("Sapphire Bot", opinion);
}
main();
```

5. Frontier LLM Self-Training Agent

An autonomous agent can use ml.distributed to self-train or fine-tune its own LLM on a GPU cluster:

```
fn autonomous_train_agent() {
    let model = ml.distributed.Transformer({"layers": 80, "hidden": 8192,
        "heads": 64, "vocab": 128000, "precision": "fp8"});
    let cluster = ml.distributed.Cluster({"gpu_type": "H100-80GB",
        "num_gpus": 512, "gpus_per_node": 8});
    let result = ml.distributed.train(model, cluster,
        {"tokens": 10000000000000, "batch_size": 2048});
    agent.memory.remember("strategy", result.strategy);
    agent.memory.remember("mfu", result.mfu_percent);
    print("Training complete. MFU={result.mfu_percent}%");
}
autonomous_train_agent();
```

6. 5D Auto-Parallelism Axes

Axis	Description
TP (Tensor)	Splits attention heads/MLP columns across GPUs within a node. Uses NVLink.
PP (Pipeline)	Assigns layers to different GPU pipeline stages. Micro-batch 1F1B schedule.

DP (Data)	Replicates model; splits batch. Gradient AllReduce after backward.
EP (Expert)	MoE expert sharding. Router sends tokens to correct expert GPU.
SP (Sequence)	Splits sequence dimension. Enables ultra-long context (1M+ tokens).
ZeRO-1/2/3	Shards optimizer states / gradients / weights across DP ranks (FSDP).

7. Kernel Stack

CUDA -> cuBLAS/cuDNN -> Fused Kernels -> FlashAttention-3 -> Triton/CUDA -> NCCL -> Topology-Aware Scheduling

FlashAttention-3: IO-optimal fused attention in SRAM (no HBM roundtrip). H100/H200 only.
FP8 TransformerEngine: 8-bit matmuls with BF16 accumulation. ~2x throughput vs BF16.
NCCL: Ring AllReduce, recursive halving, topology-aware routing. Async overlap with backward pass.